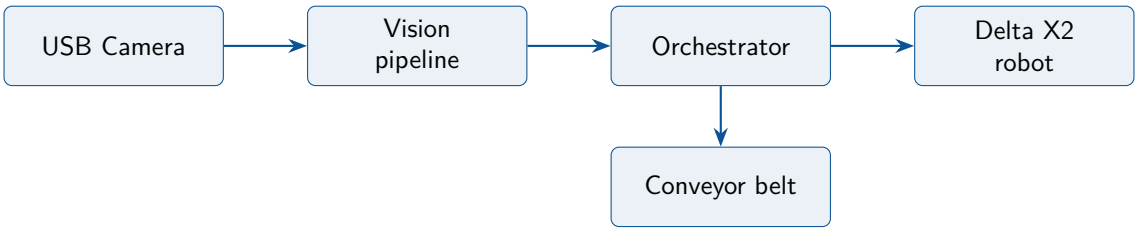
Delta X2 LEGO Sorting System

Operations Manual

Installation ▪ Configuration ▪ Operation



Software version 0.1.0
Document revision 1.1 — July 2026
Target platform: Linux (Ubuntu 24.04+)

Contents

1 System Overview

1.1 Purpose

The Delta X2 LEGO Sorting System is a Rust application that drives a Delta X2 delta robot to pick LEGO bricks from a moving conveyor belt and sort them into bins. A USB camera watches the belt upstream of the robot; a vision pipeline detects bricks, converts their pixel positions into robot coordinates, and an orchestrator schedules pick-and-place moves that are executed as G-code over a serial connection.

1.2 Hardware components

Component	Interface and protocol
Delta X2 robot (SP-X2)	USB serial (`/dev/ttyUSB0`), G-code at 115 200 baud. Workspace: X/Y $\in [-160, 160]$ mm, Z $\in [-200, 0]$ mm (Z= 0 at the top). Full protocol: Appendix A.
Conveyor belt	Separate USB serial (`/dev/ttyUSB1`); `M3 S<speed>` to start, `M5` to stop.
USB camera	UVC device via OpenCV; default 1280×720. Fixed, even lighting over the belt is required for reliable detection.
Gripper / suction	Driven through the robot controller: `M03` on, `M05` off.

Table 1.1: Hardware interfaces

1.3 Software architecture

The application is asynchronous (tokio) with a Slint GUI. It is organised in three layers plus the UI:

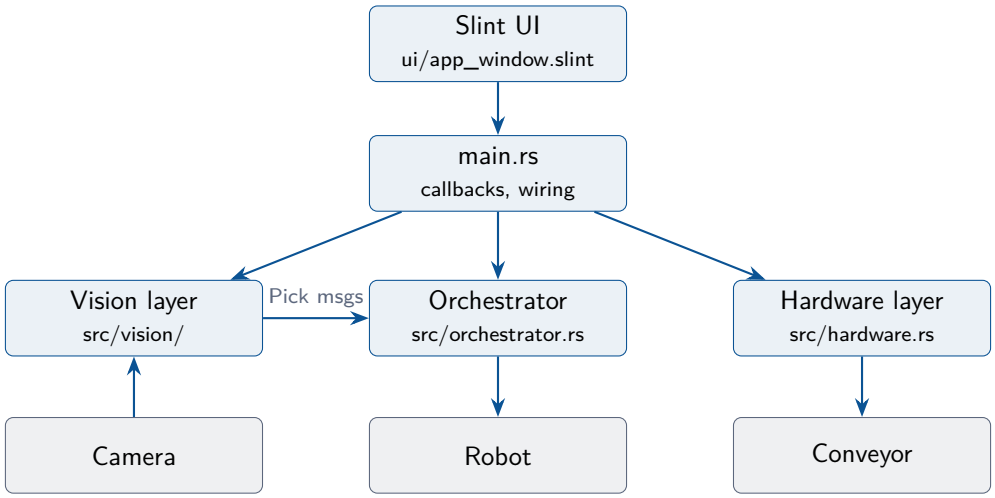


Figure 1.1: Layered architecture. Hardware access always goes through the trait abstractions of the hardware layer, so every component can also run against mock implementations (`-mock`).

Hardware layer (`src/hardware.rs`)
Async traits `RobotController`, `ConveyorController` and `CameraDriver`, each with a real and a mock implementation. The real robot driver validates every target position against the configured workspace before any G-code is sent, and synchronises on the robot's `FEEDBACK` echo so a command completes only when the physical move has finished.

Vision layer (`src/vision/`)
Blob detection (grayscale, blur, threshold, contours), pixel-to-world calibration (affine transform with rotation), a multi-frame object tracker (nearest-neighbour matching with belt-motion prediction; each physical object is reported exactly once) and a classifier. The vision pipeline owns the camera and feeds pick requests to the orchestrator; the classifier is still a placeholder (see the project TODO list).

Orchestrator (`src/orchestrator.rs`)
Receives messages (pick requests, raw commands, pause/resume, emergency stop) over a channel, maintains the instruction queue and executes commands one at a time on the robot.

UI (`ui/app_window.slint`)
Operator dashboard with start/pause, conveyor stop, homing and a prominent emergency-stop button.

1.4 Safety design

Three mechanisms protect the machine and its surroundings:

1. **Configuration validation at startup.** `Settings.toml` is checked before any hardware is touched; a pick height outside the robot's physical Z range, or a drop bin outside the workspace, aborts startup with a clear error.

2. **Workspace enforcement per move.** Every `move_to` target is validated against the configured limits inside the robot driver itself. An out-of-bounds target is rejected; it can never reach the hardware.

3. **Preemptive emergency stop.** The E-stop button writes the halt commands (`M112` to the robot, `M5` to the conveyor) on *dedicated serial handles* that bypass the normal command queue and locks, so the halt is delivered even while a move is in flight. The orchestrator queue is cleared and paused at the same time.

Safety warning

The software E-stop depends on a working USB serial link and on the robot firmware processing `M112`. It is *not* a substitute for a hardware emergency stop on the robot's power circuit. Always keep the physical E-stop within reach during operation.

2 Installation

2.1 Prerequisites

The system is designed around the following hardware and software.

2.1.1 Hardware

- **Delta X2 delta robot** (basic kit, docs.deltaxrobot.com), connected over USB serial.
- **Conveyor belt** with a G-code speed controller, connected over USB serial.
- **USB camera** mounted above the belt. The exact model is not chosen yet; all capture parameters (resolution, frame rate, pixel format) are configurable in the `[camera]` section (default 1280×720 @ 30 fps).
- **Target computer: Raspberry Pi 4 or 5** running a 64-bit Linux (ARM64/aarch64).
- **Operator display: official Raspberry Pi 7 ” Touch Display** (800×480, DSI, capacitive touch). The user interface is designed touch-first for this resolution: finger-sized controls, no scrolling on the main operator view, and the emergency-stop button always remains the largest target on screen.

2.1.2 Software

- Linux only (see *Supported platform* below); no Windows or macOS support is intended.
- Rust toolchain $\geq$ 1.85 (the crate uses the 2024 edition).
- The system packages listed in the next section (OpenCV, libclang, libudev).

Development and testing do not require the robot hardware: an x86_64 Ubuntu machine can run the full application with simulated hardware (`cargo run -- --mock`). The production build path for the Raspberry Pi (cross-compilation from the development machine versus building on the Pi itself) is not yet fixed; until it is, the repository deliberately carries no deployment tooling.

2.2 Supported platform

Linux is the supported platform; Ubuntu 24.04 or newer is recommended. Other distributions work if the packages below are available.

2.3 System packages

The build needs a C/C++ toolchain, OpenCV development headers, libclang (for generating the OpenCV Rust bindings) and libudev (for serial-port discovery):

```
sudo apt update && sudo apt install -y \
    clang libclang-dev llvm-dev \
    libudev-dev libopencv-dev pkg-config
```

Package	Why it is needed
`clang`, `libclang-dev`, `llvm-dev`	The `opencv` crate generates Rust bindings from the C++ headers at build time using libclang.
`libopencv-dev`	OpenCV itself (video capture, image processing).
`libudev-dev`	Required by the `serialport` crate for USB/serial device discovery.
`pkg-config`	Lets the build scripts locate the installed libraries.

Table 2.1: System package roles

2.4 Rust toolchain

Install the stable Rust toolchain via rustup:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2.5 Serial port permissions

On Linux the serial devices belong to the `dialout` group:

```
sudo usermod -a -G dialout $USER
# Log out and back in for the change to take effect.
```

2.6 Building and first run

```
cd deltax2sort
cargo build            # compile (also builds the Slint UI)
cargo test             # run the unit test suite
cargo run -- --mock    # start with simulated hardware
cargo run              # start against the real hardware
```

Tip

Always bring the system up in mock mode first (`cargo run - -mock`) after installing or changing the configuration. Mock mode exercises the full control flow — UI, orchestrator, queueing, E-stop — without moving any hardware.

2.7 Troubleshooting

`clang-sys` cannot find `libclang`

Install `libclang-dev`. If only a versioned library such as `libclang-21.so.1` exists, create a symlink named `libclang.so` (the repository keeps one under `libs/`) and point the build at it:

```
ln -sf /usr/lib/x86_64-linux-gnu/libclang-21.so.1 libs/libclang.so
LIBCLANG_PATH=$PWD/libs cargo build
```

`libudev-sys` build failure
: `libudev-dev` is missing; install it and rebuild.

OpenCV build failure
: Ensure `libopencv-dev` and `pkg-config` are installed and that `pkg-config --modversion opencv4` prints a version.

***Permission denied* opening `/dev/ttyUSB*`**
: Your user is not in the `dialout` group yet, or you have not logged out and back in.

Robot fails the `IsDelta` handshake
: Check the port assignment (robot vs. conveyor may be swapped), the baud rate, and that no other program holds the port open. See Appendix A for the handshake protocol.

3 Configuration

3.1 The configuration file

All settings live in a single TOML file, `Settings.toml` in the working directory by default; an alternative path can be given with `-config <path>`. If the file does not exist, a default one is created on first start.

Note

The configuration is **validated at startup**. Inconsistent values — for example a `z_pick` below the robot's physical range, or a drop position outside the workspace — abort startup with a message naming the offending key. The application never starts with a configuration that could command the robot outside its workspace.

3.2 Section `[robot]`

Key	Default	Meaning
`port_name`	/dev/ttyUSB0	Serial device of the robot controller.
`baud_rate`	115200	Serial speed (Delta X2 standard).
`home_on_connect`	true	Home (`G28`) automatically after connecting.
`x_min, x_max`	−160, 160	Allowed X range in mm.
`y_min, y_max`	−160, 160	Allowed Y range in mm.
`z_min, z_max`	−200, 0	Allowed Z range in mm (Z= 0 is the top).
`z_pick`	−180	Z height at which bricks are picked (belt surface).
`z_travel`	−50	Safe Z height for horizontal travel.
`feed_rate`	15000	G-code feed rate in mm/min (`F` parameter).
`release_gripper_on_estop`	false	Open the gripper (`M05`) just before `M112` on emergency stop. Leave false to keep a held part instead of dropping it at an arbitrary position; enable only where dropping on E-stop is safe.

Table 3.1: `[robot]` keys

Constraints enforced by validation: each axis minimum must be below its maximum; the bounds must stay within the physical envelope (X/Y ±160 mm, Z $[-200, 0]$ mm) — they may be tighter but never wider; `z_pick` and `z_travel` must lie within $[$`z_min`$,$ `z_max`$]$; `z_travel` must be above `z_pick`; and `baud_rate` and `feed_rate` must be non-zero (an `F0` feed rate would stall every move). A configuration that violates any of these is rejected at startup before any hardware is touched.

Safety warning

Reducing `x_min`/`x_max`/`y_min`/`y_max` below the physical maximum is a legitimate way to keep the robot away from fixtures near the belt. Extending them beyond the Delta X2 specification (X/Y ±160 mm, Z $[-200, 0]$ mm; see Appendix A) is now rejected by validation,

since it would let `move_to` accept unreachable targets.

3.3 Section `[conveyor]`

Key	Default	Meaning
`port_name`	/dev/ttyUSB1	Serial device of the belt controller.
`baud_rate`	115200	Serial speed.
`default_speed`	1000	Raw `S` value sent with `M3` on start.
`speed_mm_s`	100.0	Belt speed in robot coordinates, mm/s. Signed: positive means objects travel toward $+Y$. Used by the trajectory planner.

Table 3.2: `[conveyor]` keys

3.4 Section `[camera]`

Key	Default	Meaning
`device_id`	0	OpenCV device index (`/dev/video0` → 0).
`width, height`	1280, 720	Requested capture resolution.
`fps`	30	Requested capture frame rate.
`fourcc`	(unset)	Optional pixel-format FOURCC (e.g. `"MJPG"`); many UVC cameras need it to reach full frame rate at higher resolutions. Unset keeps the driver default.

Table 3.3: `[camera]` keys

The camera model is not fixed yet, so all capture parameters are configurable rather than hard-coded. The requested values are best-effort: the device may pick the nearest mode it supports, and the driver logs (and adopts) the resolution and frame rate actually in effect at connection time. Calibration is always based on the effective resolution.

3.5 Section `[sorting]`

Sorting is two independent layers. **Recognition** (the learned object catalogue) answers *what is it*; **assignment** answers *which bin does that type go to*. Keeping them separate means the same catalogue can drive different bin layouts, and you can re-map a class to another bin without re-teaching it. A class with no assignment — and any unrecognised object — is *not* picked: it stays on the belt and reaches the catch bin at the end. That "do nothing" default is what keeps an uncertain object safe.

Bins are declared as repeated `[[sorting.bins]]` tables (as many as your rig has, each at a fixed, measured position inside the workspace), and assignments as repeated `[[sorting.assignments]]` tables mapping a class name to a bin `id`.

```
[[sorting.bins]]
id = "bin-1"
x = 120.0
y = 60.0
z = -100.0
```

Key	Meaning
[[sorting.bins]] id	Stable bin identifier, referenced by assignments. Must be unique.
[[sorting.bins]] x, y, z	Drop position of that bin, in robot millimetres; must be inside the workspace.
[[sorting.assignments]] class	Learned class name to route.
[[sorting.assignments]] bin	The id of the bin it goes to (must exist).

Table 3.4: [sorting] keys

```
[[sorting.assignments]]
class = "brick-2x4-red"
bin = "bin-1"
```

3.6 Section [vision]

Key	Default	Meaning
threshold	60.0	Fixed binary threshold (0–255) applied after grayscale conversion and blur.
min_area, max_area	500, 10000	Contour area window in pixels^2 for a blob to count as an object.
invert	true	true: objects darker than the belt; false: objects lighter than the belt.
mm_per_px	0.5	Camera scale at the belt plane, millimetres per pixel. Sets the pixel-to-robot transform; measure it (e.g. with a ruler on the belt) and update it after any change to the camera height or capture resolution. Must be > 0.

Table 3.5: [vision] keys

3.7 Section [logging]

The application writes a debug log to a daily-rotating file and, by default, mirrors it to the console. The whole section is optional — omit it and the defaults below apply.

Log levels carry meaning: error marks something needing operator attention, warn a recovered or skipped condition, info normal lifecycle events, and debug high-volume detail (per-frame vision).

3.8 Section [ui]

One binary runs in two roles, selected by [ui].profile (overridable at launch with -profile):

3.9 Section [recognition]

Object recognition pairs an ONNX image embedder with a nearest-neighbour *catalogue* (the portable "learned file"): each object crop is embedded, then matched by cosine similarity against

Key	Default	Meaning
`level`	"info"	Verbosity in the `RUST_LOG` grammar: a bare level (`error`/`warn`/`info`/`debug`/`trace`) or a per-module filter such as `"info,deltax2sort=debug"` to trace G-code without flooding. A `RUST_LOG` environment variable, if set, overrides this at startup.
`directory`	"logs"	Directory the log files are written to (created if absent). One file per day; the current file is `deltax2sort_rCURRENT.log`.
`to_console`	true	Also print records to `stderr`. Useful in development and mock runs; a pure kiosk can set it `false`.
`keep_days`	7	How many rotated daily files to keep; older ones are pruned automatically. `0` keeps them all (watch disk on the Pi).

Table 3.6: `[logging]` keys

Key	Default	Meaning
`profile`	"pi"	`"pi"`: the deployed sorter — an 800×480 touch panel with sorting controls only. `"workstation"`: a larger window that also exposes the learning interface and catalogue export, for teaching the system new objects.

Table 3.7: `[ui]` keys

the labelled examples you have taught. It is *disabled by default* — nothing is recognised, and (since sorting keys on class) nothing is picked, until you generate a model and teach some classes. Recognition is independent of bins: see §3.10 and the `[sorting]` assignments for how a recognised class is routed.

Key	Default	Meaning
`enabled`	false	Master switch. While false the classifier is a stub: every object is unrecognised.
`model_path`	"models/embedder.onnx"	ONNX embedder produced by `python models/export_embedder.py` (see `models/README.md`).
`catalog_path`	"catalog.toml"	The learned catalogue. Portable — copy it from the teaching workstation to the Pi. Absent = start empty.
`match_threshold`	0.7	Minimum cosine similarity (in $[-1, 1]$) to accept a match; below it the object is left unrecognised.
`input_size`	224	Square input size the embedder expects; must match the export script.

Table 3.8: `[recognition]` keys

3.10 Camera calibration

The pixel-to-robot transform is affine: scale (mm per pixel), rotation and offset. There is deliberately no built-in default: at startup the parameters are constructed from the capture resolution actually granted by the camera and the configured `vision.mm_per_px` (the helper `CalibrationParams::centered(width, height, mm_per_px)` maps the optical centre to the robot origin, with rotation 0, i.e. the camera is assumed to be mounted with its image y axis along the belt's $+Y$ direction). A guided calibration wizard that also measures rotation and

offset is planned (see the TODO list); until it exists, mount the camera accordingly and measure `mm_per_px` by hand.

3.11 Complete example

```
[robot]
port_name = "/dev/ttyUSB0"
baud_rate = 115200
home_on_connect = true
x_min = -150.0
x_max = 150.0
y_min = -150.0
y_max = 150.0
z_min = -200.0
z_max = 0.0
z_pick = -180.0
z_travel = -150.0
feed_rate = 15000
release_gripper_on_estop = false

[conveyor]
port_name = "/dev/ttyUSB1"
baud_rate = 115200
default_speed = 1000
speed_mm_s = 100.0

[camera]
device_id = 0
width = 1280
height = 720

[[sorting.bins]]
id = "bin-1"
x = 120.0
y = 60.0
z = -100.0

[[sorting.assignments]]
class = "brick-2x4-red"
bin = "bin-1"

[vision]
threshold = 60.0
min_area = 500.0
max_area = 10000.0
invert = true
mm_per_px = 0.5

[logging]
level = "info"
directory = "logs"
to_console = true
keep_days = 7
```

4 Operation

4.1 Command line

Option	Effect
`-m`, `-mock`	Use simulated robot, conveyor and camera. No hardware required. The mock camera renders a synthetic part once per second, so the full detect → track → pick path runs (press *Start Sorting* to see the simulated robot execute the picks).
`-c`, `-config <path>`	Configuration file (default `Settings.toml`).
`-h`, `-help`	Show help.

Table 4.1: Command-line options

Logging uses `env_logger`; set `RUST_LOG` to control verbosity:

```
RUST_LOG=info cargo run -- --mock      # normal operation
RUST_LOG=debug cargo run               # includes G-code level tracing
```

4.2 Startup sequence

On start the application: loads and validates the configuration; connects to the robot (`IsDelta` handshake, Appendix A), forces absolute positioning (`G90`), and homes it if `home_on_connect` is set; connects to the conveyor; acquires the emergency-stop handles; connects to the camera; starts the orchestrator; starts the vision loop (which takes ownership of the camera and turns detected objects into pick requests); and finally opens the operator UI. Any failure in this chain aborts startup with a message identifying the failing device.

Note

The orchestrator starts **paused**: even if objects are already visible to the camera, the robot will not move until the operator presses *Start Sorting*. The status line shows *Ready (paused)* in this state.

4.3 Automatic sorting

While the belt runs, the vision loop detects objects, follows each one across several frames and hands the orchestrator exactly one pick request per object, with the position converted to robot coordinates using the calibration of section 3.10. The frame-to-frame belt prediction is applied only while the belt is actually running; when it is stopped, tracks hold their position rather than drifting, so a stationary part is never mistaken for a new object and re-requested. Two safeguards apply on the receiving side: pick requests are discarded while the orchestrator is paused (including after an emergency stop or a command failure), and any pick request older than 3 seconds is discarded as stale, since the belt will have carried the object away. Discarded picks are logged as warnings. If frame capture fails, the vision loop logs a warning and retries every 500 ms without affecting robot control.

4.4 Operator interface

Control	Behaviour
Start Sorting / Pause System	Starts the belt at `default_speed` *and* resumes the pick pipeline; pressing again stops the belt and pauses picking. Disabled after an emergency stop until the robot has been re-homed.
Stop Conveyor	Stops the belt and pauses the pick pipeline (enabled while running).
Home Robot	Homes the robot with priority — it runs even while the orchestrator is paused, and a successful homing is what clears the E-stop lockout. Available while paused or E-stopped, and disabled while actively sorting (pause first, so a home cannot preempt a live pick). Once pressed it shows *Homing…* and is locked until the orchestrator confirms the home is done.
Engage / Release Gripper	Manually opens or closes the gripper. A recovery/maintenance action — runs even while paused so a part held after a failed pick or an emergency stop can be released by hand — and is disabled while sorting so it cannot disturb an active pick.
EMERGENCY STOP	Immediately halts robot and conveyor and clears all queued work. See section 4.5.

Table 4.2: UI controls

In the *workstation* profile (§3.10 and `[ui].profile`), when recognition is on and an object is not recognised, a **learning panel** appears: it shows the object's thumbnail and the nearest known classes. The operator either assigns it to one of those classes or types a new class name and creates it; the example is added to the catalogue and saved. Unlabelled (*Skip*) objects are simply left on the belt.

Teaching saves the catalogue automatically; the workstation also has an *Export Catalogue* button that re-saves it and confirms the path. To deploy: copy the learned catalogue (`[recognition].catalog_path`) and the model (`[recognition].model_path`, i.e. `models/embedder.onnx`) to the Pi, set `[recognition].enable = true` there, and run the Pi in its default `"pi"` profile. The Pi never needs the training tools — only `tract` (built in) plus those two files.

The status line reflects the orchestrator's *confirmed* state — *Ready (paused)*, *Sorting*, or *E-STOP – HOME REQUIRED* — never merely what a button press requested. The dashboard also shows the conveyor state and the total number of sorted parts. The left panel shows the live camera feed with detection overlays: a bounding box is drawn around every object currently seen — green for a recognised part, yellow while it is still unclassified (today all parts, until the classifier is trained). The feed always shows the newest frame (older ones are dropped, so the image never lags the belt); if no frame arrives for about a second the panel switches to a red *FEED LOST* banner rather than freezing silently on the last image. *FEED LOST* is transient; if the vision pipeline itself stops (a fault in image processing), the panel shows *VISION DEAD* instead — detection is gone until the application is restarted.

When a hardware command fails — a conveyor start/stop, a robot move, or a homing — a red banner appears in the controls column with a short description, because the log is not visible on a kiosk-style touch panel. The banner stays until the operator takes an action (Start, Stop or Home), which clears it. While a conveyor command is in flight the Start/Pause button is briefly disabled (shows *Working…*) so a double tap cannot launch a second, racing command; if the command fails the conveyor status shows *ERROR* and *Stop Conveyor* stays available so the belt can be forced down.

4.5 Emergency stop and recovery

Pressing **EMERGENCY STOP**:

1. writes `M112` to the robot (optionally preceded by `M05` to open the gripper — see `release_gripper_on_esto` and `M5` to the conveyor on dedicated serial handles, bypassing every queue and lock;

2. clears the orchestrator's instruction queue and pauses it. A robot command already in flight when the halt fires is cancelled promptly (within one serial read window, not the full 30 s command timeout), because `M112` leaves its `FEEDBACK` echo unanswered — so *Home*/*Start*/*Pause* stay responsive during recovery;

3. locks out *Start Sorting* and shows *E-STOP – HOME REQUIRED* until the robot has been re-homed successfully.

To recover after an E-stop:

1. Remove the physical cause (jam, obstruction, misplaced fixture).

2. Press *Home Robot*. A successful homing clears the lockout. Note that after `M112` the robot controller may need a power cycle and an application restart before it accepts commands again, depending on firmware.

3. If a part is still held by the gripper, press *Release Gripper*. By default the gripper is *not* released automatically on E-stop, so parts are not dropped at an arbitrary position; releasing is left to this manual step once the robot is responsive again (typically after re-homing). A held part cannot be dropped by a blocking command right after `M112` anyway — the halted controller would not acknowledge it. Cells that *do* want the gripper to open on E-stop can set `release_gripper_on_estop = true`, which prepends `M05` to the halt so it opens as part of the same preemptive write.

4. Press *Start Sorting* to resume production.

Note

If a robot command fails during operation (timeout, serial error, robot fault), the orchestrator clears its queue and pauses itself automatically rather than continuing with a desynchronised state. The incident is logged with the failing command, and the gripper is released (best effort) so suction cannot keep holding a part after the pending release was discarded with the queue.

4.6 Routine operation checklist

1. Check that the belt is empty and the workspace is clear.

2. Start the application; confirm *Status: Ready (paused)*.

3. If not homed automatically, press *Home Robot*.

4. Press *Start Sorting* — this starts the belt and enables picking.

5. Supervise the first picks; keep a hand near the E-stop.

4.7 Shutdown

Closing the window shuts the system down gracefully rather than cutting power mid-motion: the conveyor is stopped, the vision loop releases the camera, and the orchestrator finishes the command in flight before parking the robot — it opens the gripper (so no part is left held under vacuum) and raises the arm to `z_travel` above its last position. Only then does the process exit. If the robot does not respond within a short timeout the application exits anyway, so a wedged controller can never hang the shutdown. When the system is stopped by an emergency stop, the park is skipped (the halted controller would ignore it); recover with *Home Robot* and, if needed, *Release Gripper*.

5 Maintenance and Development

5.1 Project layout

Path	Contents
`src/main.rs`	Entry point, UI callbacks, task wiring.
`src/hardware.rs`	Robot, conveyor and camera drivers plus mocks; workspace limits; emergency-stop handles.
`src/orchestrator.rs`	Message loop, instruction queue, trajectory planner.
`src/vision/`	Detector, calibration, tracker, classifier.
`src/app_config.rs`	Configuration structures, loading, validation.
`ui/app_window.slint`	Operator interface definition.
`docs/`	Specifications, this manual (sources under `docs/manual/`), TODO list.

Table 5.1: Repository layout

5.2 Tests

Run the unit test suite with `cargo test`. Covered today: configuration parsing and validation (including backward compatibility with older configuration files), the pixel-to-world calibration transform, the trajectory-planner intercept mathematics, workspace-limit enforcement in both robot drivers, and the instruction queue.

5.3 Development workflow

Develop against mock hardware (`-mock`) whenever possible. The mocks implement the same traits as the real drivers, including workspace-limit enforcement, so orchestration logic can be validated safely. The robot's serial protocol is documented in Appendix A.

5.4 Editing this manual

The manual sources live under `docs/manual/`, one file per chapter; `docs/manual.tex` is the driver that sets the class and includes them. Build with:

```
cd docs && latexmk -pdf manual.tex
```

5.5 Remaining work

The authoritative, maintained list of open work items lives in `docs/TODO.md`. Major open areas at the time of writing: wiring the vision loop end-to-end (camera → detection → pick scheduling), the live camera feed and overlays in the UI, a real object tracker and classifier, belt-motion compensation using robot position feedback, and a camera calibration wizard.

A Delta X2 G-code Protocol Reference

This appendix is the authoritative description of the G-code dialect and communication protocol of the Delta X2 robot as used by this project.

A.1 Communication protocol

Parameter	Value
Transport	USB serial (CDC), typically `/dev/ttyUSB*` or `/dev/ttyACM*`
Baud rate	115 200
Line ending	`\n` (newline)
Handshaking	The robot answers commands; for critical synchronisation use the `FEEDBACK` parameter (section A.4).

Table A.1: Serial link parameters

A.2 Identification and status

A.2.1 `IsDelta`

Queries the device to confirm it is a Delta X robot.

- **Response:** `YesDelta`

The application performs this handshake on every connection and refuses to proceed if the answer does not arrive.

A.2.2 `DeltaState`

Queries the current operating state of the robot.

Response	Meaning
`Free`	Idle and ready.
`Running`	Executing a command.
`Wait`	Waiting for input or synchronisation.
`Almostdone`	Near the end of a movement.
`Done`	Movement or command completed.

Table A.2: `DeltaState` responses

A.2.3 `Position`

Returns the current coordinates of the robot.

- **Response format:** `X:<val> Y:<val> Z:<val> W:<val> U:<val>`

Note

`Position` is not used by the application yet; it is the basis for the planned belt-motion compensation (robot position feedback for the trajectory planner — see `docs/TODO.md`).

A.3 Movement commands

A.3.1 `G00` / `G01` — linear movement

Moves the end-effector to the specified coordinates.

- **Syntax:** `G01 X<val> Y<val> Z<val> F<speed>`
- `F` is the feed rate in mm/min (configured via `robot.feed_rate`).

```
G01 X10.5 Y-20.0 Z-150.0 F2000
```

A.3.2 `G28` — homing

Moves all axes to their home position (top endstops).

- **Origin (X0 Y0 Z0):** after homing, the robot is positioned at the top centre.
- **Z axis:** moves downward (negative values). Z= 0 is the top; Z= −200 is near the bottom of the workspace.

A.3.3 `G90` / `G91` — positioning mode

- `G90`: interpret subsequent coordinates as absolute positions relative to the origin. The application forces this mode at the end of every connection — before any move can be issued and regardless of `home_on_connect` — and re-asserts it after homing. All `G01` moves and the workspace safety check assume absolute coordinates.
- `G91`: interpret subsequent coordinates as displacements from the current position (useful for jogging).

A.4 Synchronisation — `FEEDBACK`

`FEEDBACK:<string>` can be appended to any G-code command. The robot echoes `<string>` back on the serial port once the command has been *physically executed* — not merely received.

```
G01 X0 Y0 Z-50 FEEDBACK:done
```

Response (after the move finishes):

```
done
```

Note

The robot driver appends a unique `FEEDBACK:sync_<n>` id to every command and blocks until the echo returns. This is what makes a completed `move_to` call mean "the arm has arrived", and it is why every command has an overall timeout: a lost echo surfaces as an error instead of a hang.

A.5 End effector controls

Command	Effect
`M03`	Output on (open gripper / start vacuum).
`M05`	Output off.
`M112`	Emergency stop (used by the E-stop path).

Table A.3: M-codes

A.6 Physical workspace (SP-X2)

Axis	Range
X	−160 mm to +160 mm
Y	−160 mm to +160 mm
Z	−200 mm to 0 mm (0 at the top)
Working diameter	320 mm

Table A.4: Workspace of the SP-X2 model

These are the hard physical limits; the configured workspace (`[robot]` section, chapter on configuration) must always be equal to or smaller than them.

A.7 Quick reference

Command	Meaning
`IsDelta`	Handshake; the robot answers `YesDelta`.
`DeltaState`	Operating state query.
`Position`	Current coordinates query.
`G28`	Home all axes (to the top; X0 Y0 Z0 afterwards).
`G90` / `G91`	Absolute / relative positioning mode.
`G01 X.. Y.. Z.. F..`	Linear move at feed rate `F` (mm/min).
`M03` / `M05`	End effector (gripper/pump) on / off.
`M112`	Emergency stop.
`FEEDBACK:<id>`	Echoed back when the command has physically completed.

Table A.5: Protocol summary

Index